

Dapper and Entity Framework | JWT | IEnumerable vs IQueryable | ref vs out | Boxing vs Unboxing | Execute Scalar

Table of Contents

1. [Dapper ORM](#)
 2. [Entity Framework](#)
 3. [JWT Authentication](#)
 4. [IEnumerable vs IQueryable](#)
 5. [Ref vs Out Parameters](#)
 6. [Boxing vs Unboxing](#)
 7. [ExecuteScalar](#)
 8. [Interview Questions](#)
-

Dapper ORM

What is Dapper?

Dapper is a lightweight ORM (Object-Relational Mapper) created by Marc Gravell at Stack Overflow. It's a NuGet package that sits on top of your [ADO.NET](#) connection and provides minimal abstraction to the database. Think of it as the "fast, no-nonsense friend" of ORMs.

Key Characteristics

- **Lightweight:** Only 19KB, no dependencies
- **Fast:** One of the fastest ORMs available, close to raw [ADO.NET](#) performance
- **Simple:** Write your own SQL queries
- **Flexible:** Works with any database that [ADO.NET](#) supports

How Dapper Works

Dapper extends the IDbConnection interface with extension methods that map query results to objects automatically.

Practical Example

```
// Installation: Install-Package Dapper

using Dapper;
using System.Data;
using System.Data.SqlClient;

public class UserRepository
{
    private string _connectionString = "Server=localhost;Database=MyDb;Trusted_Connection=true;";

    // GET - Retrieve single user
    public User GetUserById(int userId)
    {
        using (IDbConnection connection = new SqlConnection(_connectionString))
        {
            string query = "SELECT UserId, UserName, Email FROM Users WHERE UserId = @userId";
            return connection.QueryFirstOrDefault<User>(query, new { UserId = userId });
        }
    }

    // GET - Retrieve multiple users
    public IEnumerable<User> GetAllUsers()
    {
        using (IDbConnection connection = new SqlConnection(_connectionString))
        {
            string query = "SELECT UserId, UserName, Email FROM Users";
            return connection.Query<User>(query);
        }
    }

    // CREATE - Insert user
    public void InsertUser(User user)
    {
        using (IDbConnection connection = new SqlConnection(_connectionString))
        {
            string query = "INSERT INTO Users (UserName, Email) VALUES (@UserName, @Email)";
            connection.Execute(query, user);
        }
    }
}
```

```

        string query = @"INSERT INTO Users (UserName, Email)
                        VALUES (@UserName, @Email)";
        connection.Execute(query, new { UserName = user.UserName, Email = user.Email });
    }
}

// UPDATE - Modify user
public void UpdateUser(User user)
{
    using (IDbConnection connection = new SqlConnection(_connectionString))
    {
        string query = @"UPDATE Users
                        SET UserName = @UserName, Email = @Email
                        WHERE UserId = @UserId";
        connection.Execute(query, user);
    }
}

// DELETE - Remove user
public void DeleteUser(int userId)
{
    using (IDbConnection connection = new SqlConnection(_connectionString))
    {
        string query = "DELETE FROM Users WHERE UserId = @UserId";
        connection.Execute(query, new { UserId = userId });
    }
}

public class User
{
    public int UserId { get; set; }
    public string UserName { get; set; }
    public string Email { get; set; }
}

```

Dapper Methods Explained

- **Query():** Returns multiple rows as IEnumerable
- **QueryFirstOrDefault():** Returns first row or null

- **QuerySingleOrDefault():** Returns single row or null (throws if multiple)
- **Execute():** Executes a command without returning data
- **ExecuteScalar():** Returns a single value from first column, first row

Pros and Cons

Pros:

- Extremely fast
- Minimal overhead
- Full control over SQL
- Easy to learn

Cons:

- Write SQL manually
 - No query validation at compile time
 - Less abstraction means more code
 - No built-in navigation properties
-

Entity Framework

What is Entity Framework?

Entity Framework (EF) is a full-featured ORM created by Microsoft. It's a high-level abstraction layer that allows you to work with databases using objects instead of writing SQL. Think of it as the "intelligent friend" that understands relationships and handles complexity.

Entity Framework Versions

- **Entity Framework 6:** Last version of classic EF (older .NET Framework)
- **Entity Framework Core:** Modern version for .NET Core and .NET 5+

Core Concepts

DbContext: Your gateway to the database

DbSet: Represents a table in the database

Models: Classes that represent database tables

Migrations: Version control for your database schema

Practical Example

```
// Installation: Install-Package Microsoft.EntityFrameworkCore
// Install-Package Microsoft.EntityFrameworkCore.SqlServer

using Microsoft.EntityFrameworkCore;

// Define your DbContext
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }

    public DbSet<User> Users { get; set; }
    public DbSet<Post> Posts { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder.UseSqlServer("Server=localhost;Database=MyDb;Trusted_Connection=true");
        }
    }
}

// Define your Models
public class User
{
    public int UserId { get; set; }
    public string UserName { get; set; }
    public string Email { get; set; }

    // Navigation property
    public ICollection<Post> Posts { get; set; } = new List<Post>();
}

public class Post
{
    public int PostId { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
```

```

public int UserId { get; set; }

// Navigation property
public User User { get; set; }
}

// Repository Pattern with EF
public class UserRepository
{
    private readonly ApplicationDbContext _context;

    public UserRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET - Retrieve user with posts
    public User GetUserWithPosts(int userId)
    {
        return _context.Users
            .Include(u => u.Posts) // Eager loading
            .FirstOrDefault(u => u.UserId == userId);
    }

    // GET - All users
    public List<User> GetAllUsers()
    {
        return _context.Users.ToList();
    }

    // CREATE - Add user
    public void AddUser(User user)
    {
        _context.Users.Add(user);
        _context.SaveChanges();
    }

    // UPDATE - Modify user
    public void UpdateUser(User user)
    {

```

```

        _context.Users.Update(user);
        _context.SaveChanges();
    }

    // DELETE - Remove user
    public void DeleteUser(int userId)
    {
        var user = _context.Users.Find(userId);
        if (user != null)
        {
            _context.Users.Remove(user);
            _context.SaveChanges();
        }
    }

    // Complex query with filtering and sorting
    public List<User> GetActiveUsersWithPostCount()
    {
        return _context.Users
            .Where(u => u.Posts.Count > 0)
            .OrderByDescending(u => u.Posts.Count)
            .ToList();
    }
}

```

Entity Framework Conventions

EF uses conventions to configure your models automatically:

- Class name = Table name
- Property name = Column name
- Properties with "Id" or "EntityNameId" are primary keys
- Navigation properties establish relationships

Key Features

Lazy Loading: Data loaded when accessed

```

var user = _context.Users.First(); // User loaded
var posts = user.Posts; // Posts loaded here (lazy)

```

Eager Loading: Data loaded with parent

```
var user = _context.Users.Include(u => u.Posts).First();
var posts = user.Posts; // Already loaded
```

Explicit Loading: Control when to load

```
var user = _context.Users.First();
_context.Entry(user).Collection(u => u.Posts).Load();
```

Pros and Cons

Pros:

- Automatically generates SQL
- Type-safe queries
- Navigation properties
- Migration support
- Relationship management

Cons:

- Slower than raw [ADO.NET](#)
- Can generate inefficient SQL
- Steep learning curve
- More memory overhead

Entity Framework vs Dapper

Feature	Entity Framework	Dapper
Query Speed	Good	Excellent
Ease of Use	Medium	Easy
Learning Curve	Steep	Shallow
SQL Control	Limited	Full
Navigation Properties	Yes	No
Change Tracking	Automatic	Manual

Best For	Complex applications	Simple, fast queries
----------	----------------------	----------------------

JWT Authentication

What is JWT?

JWT stands for JSON Web Token. It's a compact, URL-safe means of representing claims to be transferred between two parties. It's commonly used for stateless authentication in APIs.

JWT Structure

A JWT consists of three parts separated by dots (.):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG.
```

1. **Header:** Contains token type (JWT) and hashing algorithm (HS256, RS256)
2. **Payload:** Contains claims (user data). Can be decoded but NOT encrypted
3. **Signature:** Ensures token hasn't been tampered with

Practical Implementation

```
// Installation: Install-Package System.IdentityModel.Tokens.Jwt
```

```
using System;
using System.Collections.Generic;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public class JwtTokenService
{
    private readonly string _secretKey;
    private readonly string _issuer;
    private readonly string _audience;
    private readonly int _expirationMinutes;
```

```

public JwtTokenService(string secretKey, string issuer, string audience, int expirationM
{
    _secretKey = secretKey;
    _issuer = issuer;
    _audience = audience;
    _expirationMinutes = expirationMinutes;
}

// Generate JWT Token
public string GenerateToken(string userId, string userName, string email)
{
    // Create key
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_secretKey));
    var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    // Create claims
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, userId),
        new Claim(ClaimTypes.Name, userName),
        new Claim(ClaimTypes.Email, email),
        new Claim("CustomClaim", "CustomValue")
    };

    // Create token
    var token = new JwtSecurityToken(
        issuer: _issuer,
        audience: _audience,
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(_expirationMinutes),
        signingCredentials: credentials
    );

    // Return token string
    return new JwtSecurityTokenHandler().WriteToken(token);
}

// Validate and read JWT Token
public ClaimsPrincipal ValidateToken(string token)
{

```

```

try
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_secretKey));
    var tokenHandler = new JwtSecurityTokenHandler();

    var principal = tokenHandler.ValidateToken(token, new TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = key,
        ValidateIssuer = true,
        ValidIssuer = _issuer,
        ValidateAudience = true,
        ValidAudience = _audience,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.Zero
    }, out SecurityToken validatedToken);

    return principal;
}
catch
{
    return null;
}
}

// Extract user info from token
public string GetUserIdFromToken(string token)
{
    var principal = ValidateToken(token);
    return principal?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
}
}

// Usage in API Controller
public class AuthController : ControllerBase
{
    private readonly JwtTokenService _jwtService;

    public AuthController(JwtTokenService jwtService)
    {

```

```

        _jwtService = jwtService;
    }

    [HttpPost("login")]
    public IActionResult Login(LoginRequest request)
    {
        // Validate credentials (simplified)
        if (request.Username == "admin" && request.Password == "password")
        {
            var token = _jwtService.GenerateToken("1", "admin", "admin@example.com");
            return Ok(new { token });
        }

        return Unauthorized();
    }

    [HttpGet("protected")]
    [Authorize] // Requires valid JWT token
    public IActionResult ProtectedRoute()
    {
        var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        return Ok(new { message = $"Hello, user {userId}" });
    }
}

public class LoginRequest
{
    public string Username { get; set; }
    public string Password { get; set; }
}

// Configure JWT in Startup.cs or Program.cs
public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {
        var jwtSettings = new JwtSettings
        {
            SecretKey = "your-super-secret-key-at-least-32-characters-long",
            Issuer = "YourAppName",

```

```

        Audience = "YourAppUsers",
        ExpirationMinutes = 60
    };

    services.AddSingleton(new JwtTokenService(
        jwtSettings.SecretKey,
        jwtSettings.Issuer,
        jwtSettings.Audience,
        jwtSettings.ExpirationMinutes
    ));

    // Add Authentication
    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options =>
        {
            options.TokenValidationParameters = new TokenValidationParameters
            {
                ValidateIssuerSigningKey = true,
                IssuerSigningKey = new SymmetricSecurityKey(
                    Encoding.UTF8.GetBytes(jwtSettings.SecretKey)),
                ValidateIssuer = true,
                ValidIssuer = jwtSettings.Issuer,
                ValidateAudience = true,
                ValidAudience = jwtSettings.Audience,
                ValidateLifetime = true
            };
        });

    services.AddControllers();
}

public void Configure(IApplicationBuilder app)
{
    app.UseRouting();
    app.UseAuthentication();
    app.UseAuthorization();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapControllers();
    });
}

```

```
});  
}  
}
```

JWT Claims Explained

Claims are statements about the user:

- **Standard Claims:** iss, sub, aud, exp, nbf, iat, jti
- **Custom Claims:** Your application-specific data

Security Best Practices

1. Store secret key securely (use environment variables or Azure Key Vault)
2. Use HTTPS always when transmitting tokens
3. Set appropriate expiration times
4. Never put sensitive data in payload (it's encoded, not encrypted)
5. Validate token signature on every request
6. Implement token refresh mechanism

Pros and Cons

Pros:

- Stateless (no server-side session storage)
- Scalable across multiple servers
- Works well with APIs and microservices
- Self-contained with all needed info

Cons:

- Token can't be invalidated immediately
- Payload is encoded, not encrypted
- Requires more careful key management
- Larger than session cookies

IEnumerable vs IQueryable

Understanding the Difference

These two interfaces are similar but fundamentally different in how they work.

IEnumerable: Works with in-memory collections

IQueryable: Works with out-of-memory data sources (databases)

Technical Explanation

```
// IEnumerable
public interface IEnumerable
{
    IEnumerator GetEnumerator();
}

// IQueryable
public interface IQueryable : IEnumerable
{
    Type ElementType { get; }
    Expression Expression { get; }
    IQueryProvider Provider { get; }
    IEnumerator IEnumerable.GetEnumerator();
    IEnumerator<T> GetEnumerator();
}
```

IEnumerable uses **System.Linq** (LINQ to Objects)

IQueryable uses **System.Linq.Expressions** (LINQ Providers)

Practical Examples

```
using System;
using System.Collections.Generic;
using System.Linq;

public class ComparisonExample
{
    // Sample data
    private List<User> users = new List<User>
    {
        new User { Id = 1, Name = "Alice", Age = 28, Active = true },
        new User { Id = 2, Name = "Bob", Age = 22, Active = false },
        new User { Id = 3, Name = "Charlie", Age = 35, Active = true },
    }
```

```

    new User { Id = 4, Name = "Diana", Age = 29, Active = true }
};

// Example 1: IEnumerable
public void IEnumerableExample()
{
    IEnumerable<User> query = users
        .Where(u => u.Age > 25) // Filtering happens in memory
        .OrderBy(u => u.Name); // Sorting happens in memory

    // When you iterate, it executes LINQ to Objects
    foreach (var user in query)
    {
        Console.WriteLine(user.Name);
    }
}

// Example 2: IQueryable
public void IQueryableExample(DbContext context)
{
    IQueryable<User> query = context.Users
        .Where(u => u.Age > 25) // Translated to SQL
        .OrderBy(u => u.Name); // Translated to SQL

    // When you call ToList() or iterate, SQL executes at database
    List<User> results = query.ToList();
}

// Example 3: Performance Difference
public void PerformanceDifference(DbContext context)
{
    // WRONG: IEnumerable (Loads ALL users, filters in memory)
    IEnumerable<User> badQuery = context.Users // All records loaded
        .Where(u => u.Age > 25); // Filtered in memory

    // RIGHT: IQueryable (Database filters before loading)
    IQueryable<User> goodQuery = context.Users
        .Where(u => u.Age > 25); // Filtered in database

    // If you have 1 million users, badQuery loads all 1 million!
}

```



```
}
```

```
// Example 4: When filtering happens
```

```
public void FilteringTiming()
```

```
{
```

```
    // IEnumerable - Deferred Execution
```

```
    IEnumerable<User> enumQuery = users
```

```
        .Where(u => u.Active) // Not executed yet
```

```
        .OrderBy(u => u.Age); // Not executed yet
```

```
    // Execution happens here when iterating
```

```
    foreach (var user in enumQuery)
```

```
    {
```

```
        Console.WriteLine(user.Name);
```

```
    }
```

```
    // IQueryable - Also Deferred Execution but at database
```

```
    IQueryable<User> queryableQuery = context.Users
```

```
        .Where(u => u.Active); // Expression tree built, not executed
```

```
    // Execution happens here
```

```
    var results = queryableQuery.ToList();
```

```
}
```

```
// Example 5: Chaining operations
```

```
public void ChainingOperations(DbContext context)
```

```
{
```

```
    var query = context.Users
```

```
        .Where(u => u.Age > 25)
```

```
        .Where(u => u.Active)
```

```
        .OrderBy(u => u.Name)
```

```
        .Select(u => new { u.Name, u.Age });
```

```
    // Single SQL query executes when calling ToList()
```

```
    var results = query.ToList();
```

```
    // SQL generated might look like:
```

```
    // SELECT Name, Age FROM Users
```

```
    // WHERE Age > 25 AND Active = 1
```

```
    // ORDER BY Name
```

```
    }  
}  
  
public class User  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public int Age { get; set; }  
    public bool Active { get; set; }  
}
```

Key Differences Summary

Aspect	IEnumerable	IQueryable
Namespace	System.Linq	System.Linq
Works with	In-memory collections	Any data source
Implementation	IEnumerator	IQueryProvider + Expression
Filtering	LINQ to Objects	Translated to source query language
Performance	Slower for large datasets	Faster (filters at source)
Methods	Standard LINQ methods	Extension methods
Example use	Arrays, Lists	Databases (Entity Framework)
Deferred Execution	Yes	Yes

When to Use

Use IEnumerable when:

- Working with in-memory collections (List, Array)
- You need simple filtering/sorting
- Performance is not critical
- Data already loaded in memory

Use IQueryable when:

- Querying databases (Entity Framework, LINQ to SQL)
- Working with large datasets

- You want server-side filtering
 - Performance is critical
-

Ref vs Out Parameters

Understanding Reference Parameters

Both `ref` and `out` allow passing parameters by reference instead of by value. They're often confused, but they have important differences.

Ref Parameter

A `ref` parameter:

- **Must be initialized** before passing
- **Can be read and modified** inside the method
- **Changes are visible** to the caller
- **Passes reference** to existing variable

```
public class RefExample
{
    // Method using ref
    public void AddBonus(ref int salary, int bonusAmount)
    {
        // salary already has a value
        salary = salary + bonusAmount; // Modified
    }

    // Usage
    public void DemoRef()
    {
        int employeeSalary = 50000;
        Console.WriteLine($"Before: {employeeSalary}"); // 50000

        AddBonus(ref employeeSalary, 10000);

        Console.WriteLine($"After: {employeeSalary}"); // 60000
    }
}
```

```

// Swap two values using ref
public void SwapValues(ref int a, ref int b)
{
    int temp = a;
    a = b;
    b = temp;
}

public void DemoSwap()
{
    int x = 10, y = 20;
    SwapValues(ref x, ref y);
    // x = 20, y = 10
}

// Modifying object properties through ref
public void ModifyUser(ref User user)
{
    user.Name = "Updated Name"; // Valid
    user = new User { Name = "New User" }; // Also valid - reassigns reference
}
}

```

Out Parameter

An `out` parameter:

- **Need NOT be initialized** before passing
- **Must be assigned** inside the method
- **Changes are visible** to the caller
- **Used primarily for returning multiple values**

```

public class OutExample
{
    // Method using out for returning multiple values
    public void GetUserInfo(int userId, out string name, out int age)
    {
        // Must assign both before returning
        name = "John Doe";
        age = 30;
    }
}

```

```
}

// Usage
public void DemoOut()
{
    // name and age don't need initialization
    GetUserInfo(1, out string name, out int age);
    Console.WriteLine($"{name}, Age {age}"); // John Doe, Age 30
}

// Parse method returning both success and value
public bool TryParseInt(string value, out int result)
{
    return int.TryParse(value, out result);
}

public void DemoTryParse()
{
    if (TryParseInt("123", out int number))
    {
        Console.WriteLine($"Parsed: {number}"); // 123
    }
}

// Real-world example: Divide with error handling
public bool DivideSafely(int dividend, int divisor, out int quotient, out int remainder)
{
    quotient = 0;
    remainder = 0;

    if (divisor == 0)
    {
        return false; // Division by zero
    }

    quotient = dividend / divisor;
    remainder = dividend % divisor;
    return true;
}
```

```

public void DemoDivide()
{
    if (DivideSafely(10, 3, out int q, out int r))
    {
        Console.WriteLine($"10 / 3 = {q} remainder {r}"); // 3 remainder 1
    }
}
}

```

Comparison

```

public class ComparisonExample
{
    // REF: Must be initialized, can read and write
    public void RefDemo()
    {
        int value = 10; // Must initialize
        RefMethod(ref value); // Pass as ref
        // value is now modified
    }

    public void RefMethod(ref int x)
    {
        x = x + 5; // Can read and modify
    }

    // OUT: Need not initialize, must assign
    public void OutDemo()
    {
        OutMethod(out int value); // Don't need to initialize
        // value is assigned by the method
    }

    public void OutMethod(out int x)
    {
        x = 15; // Must assign before returning
    }

    // Comparison table

```

```

public void ComparisonTable()
{
    /*
    Feature      | Ref   | Out
    Must initialize | Yes   | No
    Can read      | Yes   | No (not until assigned)
    Can modify     | Yes   | Yes
    Must assign    | No    | Yes
    Common use     | Swaps | Multiple returns
    */
}
}

```

Real-World Examples

```

public class RealWorldExamples
{
    // REF: Useful when passing an object you want to update
    public void UpdateUser(ref User user)
    {
        user.Name = "Updated";
        user.Email = "new@example.com";
    }

    // OUT: Useful for TryXxx patterns
    public bool TryGetUser(int id, out User user)
    {
        user = null;
        var users = new List<User> { new User { Id = 1, Name = "Alice" } };

        var found = users.FirstOrDefault(u => u.Id == id);
        if (found != null)
        {
            user = found;
            return true;
        }
        return false;
    }
}

```

```

// OUT: Multiple return values
public void SearchDatabase(string query, out List<User> results, out int totalCount, out bool hasError)
{
    results = new List<User>();
    totalCount = 0;
    hasError = false;

    try
    {
        // Search logic
        results = new List<User> { new User { Id = 1, Name = "Alice" } };
        totalCount = results.Count;
    }
    catch
    {
        hasError = true;
    }
}

public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}

```

Key Differences

Aspect	Ref	Out
Initialization before	Required	Not required
Must assign inside method	No	Yes
Can read inside method	Yes	No (until assigned)
Multiple values return	No	Yes
Common pattern	Modifying existing data	TryXxx pattern
Example	Swap operation	int.TryParse

Boxing vs Unboxing

Understanding Value and Reference Types

Value Types (int, double, bool, struct):

- Stored on the stack
- Contains actual value
- Fast access

Reference Types (class, string, object):

- Stored on the heap
- Contains reference to value
- Slower access

What is Boxing?

Boxing is the process of converting a value type to a reference type (wrapping it in an object).

```
public class BoxingExample
{
    public void BasicBoxing()
    {
        // Value type
        int age = 25; // Stored on stack

        // Boxing: int value wrapped in object (on heap)
        object boxedAge = age; // age copied to heap

        // Boxing happens implicitly here
        object boxedNumber = 5; // Creates box on heap
    }
}
```

What is Unboxing?

Unboxing is the reverse process: extracting the value type from the boxed object.

```

public class UnboxingExample
{
    public void BasicUnboxing()
    {
        // Boxing
        object boxedValue = 100;

        // Unboxing: Extract value from object back to value type
        int value = (int)boxedValue; // Must cast to correct type

        // IMPORTANT: Must match original type!
        object boxedDouble = 3.14;
        // int x = (int)boxedDouble; // Runtime error!
        // double y = (double)boxedDouble; // Correct
    }
}

```

Practical Examples

```

using System;
using System.Collections.Generic;

public class BoxingUnboxingExamples
{
    // Example 1: Implicit boxing with ArrayList (old collection)
    public void LegacyCollectionBoxing()
    {
        // ArrayList stores objects, so values get boxed
        var list = new System.Collections.ArrayList();

        list.Add(10); // Boxing: int -> object
        list.Add(20); // Boxing: int -> object
        list.Add("Hello"); // No boxing (already reference type)

        // Unboxing
        int first = (int)list[0]; // Unboxing: object -> int
        int second = (int)list[1]; // Unboxing
    }
}

```

```
// Example 2: Generic collections avoid boxing
public void GenericCollectionNoBoxing()
{
    // List<T> doesn't box value types
    var numbers = new List<int>();

    numbers.Add(10); // NO boxing
    numbers.Add(20); // NO boxing
    numbers.Add(30); // NO boxing

    int value = numbers[0]; // NO unboxing
}

// Example 3: Nullable types (useful for databases)
public void NullableBoxing()
{
    int? nullableInt = 5; // Struct, value type

    // Boxing nullable type
    object boxed = nullableInt;

    // Unboxing back
    int? unboxed = (int?)boxed;
}

// Example 4: Boxing with interfaces
public void BoxingWithInterfaces()
{
    int value = 100;

    // Boxing happens when assigning to interface
    IComparable boxed = value;
    // value is boxed to allow IComparable methods

    // This is implicit boxing
    IEquatable<int> equatable = value;
}

// Example 5: Performance impact
public void PerformanceComparison()
```

```

{
    // Boxing is expensive operation
    // Avoid in tight loops!

    // SLOW: Repeated boxing in loop
    object result = 0;
    for (int i = 0; i < 1000000; i++)
    {
        result = i; // Boxing happens 1,000,000 times!
    }

    // FAST: No boxing
    int fastResult = 0;
    for (int i = 0; i < 1000000; i++)
    {
        fastResult = i; // No boxing
    }
}

// Example 6: Common boxing scenarios
public void CommonScenarios()
{
    // 1. String concatenation
    int number = 5;
    string text = "Value: " + number; // Boxing!
    string better = $"Value: {number}"; // Also boxes, but cleaner

    // 2. Console.WriteLine with value types
    Console.WriteLine(10); // Boxing
    Console.WriteLine("Value: {0}", 10); // Boxing

    // 3. Passing value type to object parameter
    PrintObject(20); // Boxing happens

    // 4. Dictionary<K,V> with value type values
    var dict = new Dictionary<string, object>();
    dict["age"] = 25; // Boxing
    dict["name"] = "John"; // No boxing (string is reference type)
}

```

```

public void PrintObject(object obj)
{
    Console.WriteLine(obj);
}

// Example 7: Boxing with structs
public struct Point
{
    public int X { get; set; }
    public int Y { get; set; }
}

public void StructBoxing()
{
    Point p = new Point { X = 10, Y = 20 };

    // Boxing struct
    object boxedPoint = p;

    // Unboxing struct
    Point unboxedPoint = (Point)boxedPoint;
}
}

```

Memory Visualization

VALUE TYPE (int):	BOXING:
Stack Memory	Heap Memory (boxed object)
value:25	object
	value:25
	Stack points to heap
UNBOXING:	
object -> extract to new location	
Heap Memory	Stack Memory

object value:25

value:25

Key Points

1. **Boxing:** Value type -> Reference type (heap allocation)
2. **Unboxing:** Reference type -> Value type (heap extraction)
3. **Cost:** Memory allocation + performance overhead
4. **Avoidance:** Use generics instead of object type
5. **Type Safety:** Must unbox to exact original type

Best Practices

```
public class BestPractices
{
    // AVOID: Boxing in loops
    public void AvoidBoxingInLoops()
    {
        // Bad
        object sum = 0;
        for (int i = 0; i < 1000000; i++)
        {
            sum = (int)sum + i; // Unbox + Box = inefficient
        }

        // Good
        int sum2 = 0;
        for (int i = 0; i < 1000000; i++)
        {
            sum2 += i;
        }
    }

    // PREFER: Generics over object
    public void PreferGenerics()
```

```

{
    // Bad: Boxes integers
    var list = new System.Collections.ArrayList();
    list.Add(1);
    list.Add(2);
    int first = (int)list[0]; // Unboxing

    // Good: No boxing
    var genericList = new List<int>();
    genericList.Add(1);
    genericList.Add(2);
    int first2 = genericList[0]; // No boxing
}
}

```

ExecuteScalar

What is ExecuteScalar?

ExecuteScalar executes a SQL command and returns a single value (the first column of the first row). It's used when you expect a scalar result: count, sum, max, single value, etc.

Where It's Used

- COUNT queries
- SUM, AVG, MAX, MIN calculations
- ID of inserted record (IDENTITY)
- Boolean checks (exists, any)

With ADO.NET

```

using System;
using System.Data;
using System.Data.SqlClient;

public class ExecuteScalarWithAdoNet
{
    private string _connectionString = "Server=localhost;Database=MyDb;Trusted_Connecti

```

// Example 1: Count users

```
public int GetUserCount()
```

```
{
```

```
    using (SqlConnection connection = new SqlConnection(_connectionString))
```

```
    {
```

```
        string query = "SELECT COUNT(*) FROM Users";
```

```
        SqlCommand command = new SqlCommand(query, connection);
```

```
        connection.Open();
```

```
        object result = command.ExecuteScalar(); // Returns object
```

```
        // Convert to int
```

```
        return result != null ? Convert.ToInt32(result) : 0;
```

```
    }
```

```
}
```

// Example 2: Get average salary

```
public decimal GetAverageSalary()
```

```
{
```

```
    using (SqlConnection connection = new SqlConnection(_connectionString))
```

```
    {
```

```
        string query = "SELECT AVG(Salary) FROM Employees";
```

```
        SqlCommand command = new SqlCommand(query, connection);
```

```
        connection.Open();
```

```
        object result = command.ExecuteScalar();
```

```
        return result != null && result != DBNull.Value
```

```
            ? Convert.ToDecimal(result)
```

```
            : 0m;
```

```
    }
```

```
}
```

// Example 3: Get maximum ID of inserted record

```
public int InsertAndGetId(string name, string email)
```

```
{
```

```
    using (SqlConnection connection = new SqlConnection(_connectionString))
```

```
    {
```

```
        string query = @"INSERT INTO Users (UserName, Email)
```



```
VALUES (@Name, @Email);  
SELECT SCOPE_IDENTITY(); // Get inserted ID
```

```
SqlCommand command = new SqlCommand(query, connection);  
command.Parameters.AddWithValue("@Name", name);  
command.Parameters.AddWithValue("@Email", email);
```

```
connection.Open();  
object result = command.ExecuteScalar();
```

```
return result != null ? Convert.ToInt32(result) : 0;
```

```
}  
}
```

```
// Example 4: Check if user exists
```

```
public bool UserExists(string email)
```

```
{
```

```
    using (SqlConnection connection = new SqlConnection(_connectionString))
```

```
    {
```

```
        string query = "SELECT COUNT(*) FROM Users WHERE Email = @Email";
```

```
        SqlCommand command = new SqlCommand(query, connection);
```

```
        command.Parameters.AddWithValue("@Email", email);
```

```
        connection.Open();
```

```
        int count = (int)command.ExecuteScalar();
```

```
        return count > 0;
```

```
    }
```

```
}
```

```
// Example 5: Get specific value
```

```
public string GetUserNameById(int userId)
```

```
{
```

```
    using (SqlConnection connection = new SqlConnection(_connectionString))
```

```
    {
```

```
        string query = "SELECT UserName FROM Users WHERE UserId = @UserId";
```

```
        SqlCommand command = new SqlCommand(query, connection);
```

```
        command.Parameters.AddWithValue("@UserId", userId);
```

```
        connection.Open();
```

```

        object result = command.ExecuteScalar();

        return result != null ? result.ToString() : null;
    }
}

```

With Dapper

```

using Dapper;
using System.Data;
using System.Data.SqlClient;

public class ExecuteScalarWithDapper
{
    private string _connectionString = "Server=localhost;Database=MyDb;Trusted_Connection=true;";

    // Example 1: Count users
    public int GetUserCount()
    {
        using (IDbConnection connection = new SqlConnection(_connectionString))
        {
            string query = "SELECT COUNT(*) FROM Users";
            return connection.ExecuteScalar<int>(query);
        }
    }

    // Example 2: Sum of salaries
    public decimal GetTotalSalary()
    {
        using (IDbConnection connection = new SqlConnection(_connectionString))
        {
            string query = "SELECT SUM(Salary) FROM Employees";
            var result = connection.ExecuteScalar<decimal?>(query);
            return result ?? 0m;
        }
    }

    // Example 3: Get inserted ID

```

```

public int InsertAndGetId(string name, string email)
{
    using (IDbConnection connection = new SqlConnection(_connectionString))
    {
        string query = @"INSERT INTO Users (UserName, Email)
                        VALUES (@Name, @Email);
                        SELECT CAST(SCOPE_IDENTITY() as int)";

        return connection.ExecuteScalar<int>(query, new { Name = name, Email = email });
    }
}

// Example 4: Generic ExecuteScalar
public T GetScalarValue<T>(string query, object parameters = null)
{
    using (IDbConnection connection = new SqlConnection(_connectionString))
    {
        return connection.ExecuteScalar<T>(query, parameters);
    }
}

// Usage
public void DemoUsage()
{
    int count = GetUserCount();
    decimal total = GetTotalSalary();
    int newId = InsertAndGetId("John", "john@example.com");

    var maxAge = GetScalarValue<int>("SELECT MAX(Age) FROM Users");
}
}

```

With Entity Framework

```

using Microsoft.EntityFrameworkCore;

public class ExecuteScalarWithEF
{
    private readonly ApplicationDbContext _context;
}

```

```
public ExecuteScalarWithEF(ApplicationDbContext context)
{
    _context = context;
}

// Example 1: Count
public int GetUserCount()
{
    return _context.Users.Count();
}

// Example 2: Sum
public decimal GetTotalSalary()
{
    return _context.Employees.Sum(e => e.Salary);
}

// Example 3: Max
public int GetMaxAge()
{
    return _context.Users.Max(u => u.Age);
}

// Example 4: Average
public double GetAverageAge()
{
    return _context.Users.Average(u => u.Age);
}

// Example 5: Exists
public bool UserExists(string email)
{
    return _context.Users.Any(u => u.Email == email);
}

// Example 6: Get single value
public string GetUserNameById(int userId)
{
    return _context.Users
```

```

        .Where(u => u.UserId == userId)
        .Select(u => u.UserName)
        .FirstOrDefault();
    }

    // Example 7: Raw SQL with ExecuteScalar
    public int ExecuteCustomQuery(string sqlQuery)
    {
        return _context.Database
            .SqlQueryRaw<int>(sqlQuery)
            .FirstOrDefault();
    }
}

```

Return Type Handling

```

public class ReturnTypeInfo
{
    public void HandlingDifferentTypes()
    {
        // int
        int count = GetScalarValue<int>("SELECT COUNT(*) FROM Users");

        // string
        string name = GetScalarValue<string>("SELECT TOP 1 UserName FROM Users");

        // decimal
        decimal salary = GetScalarValue<decimal>("SELECT AVG(Salary) FROM Employees");

        // nullable (for NULL values in database)
        int? maxAge = GetScalarValue<int?>("SELECT MAX(Age) FROM Users");

        // bool (though uncommon)
        bool exists = GetScalarValue<bool>("SELECT CASE WHEN COUNT(*) > 0 THEN 1 ELSE 0");
    }

    private T GetScalarValue<T>(string query)
    {
        // Implementation here
    }
}

```

```
        return default;
    }
}
```

Common ExecuteScalar Queries

```
-- COUNT - Total records
SELECT COUNT(*) FROM Users

-- SUM - Total amount
SELECT SUM(Amount) FROM Orders

-- AVG - Average value
SELECT AVG(Salary) FROM Employees

-- MAX - Maximum value
SELECT MAX(Age) FROM Users

-- MIN - Minimum value
SELECT MIN(Price) FROM Products

-- EXISTS - Check existence
SELECT CASE WHEN EXISTS (SELECT 1 FROM Users WHERE Email = @Email) THEN 1 ELSE 0 END

-- SCOPE_IDENTITY - Last inserted ID
INSERT INTO Users (Name) VALUES (@Name);
SELECT CAST(SCOPE_IDENTITY() as int)

-- Check single value
SELECT Email FROM Users WHERE UserId = @UserId

-- Boolean check
SELECT CASE WHEN Age > 18 THEN 1 ELSE 0 END FROM Users WHERE UserId = @UserId
```

Performance Comparison

```
public class PerformanceComparison
{
```

```

// ExecuteScalar - Most efficient for single value
public int GetCountExecuteScalar()
{
    // Returns only the count value
    // Database returns 1 value
    // Network transfer: minimal
}

// vs ExecuteReader - Overkill for single value
public int GetCountExecuteReader()
{
    // Returns entire result set (even if 1 row)
    // More overhead than needed
}

// vs Query (LINQ) - Less efficient for aggregates
public int GetCountLinq()
{
    // Loads objects into memory
    // Then counts them
    // Wasteful if you only need count
}
}

```

Best Practices

```

public class BestPractices
{
    // 1. Always check for NULL/DBNull
    public decimal SafeGetDecimal(string query)
    {
        var result = ExecuteScalar<decimal?>(query);
        return result ?? 0m;
    }

    // 2. Use parameterized queries (prevent SQL injection)
    public int CountUsersByStatus(string status)
    {
        string query = "SELECT COUNT(*) FROM Users WHERE Status = @Status";
    }
}

```

```

        // Pass parameters separately, not in query string
        return ExecuteScalar<int>(query, new { Status = status });
    }

    // 3. Handle type mismatches carefully
    public int SafeConvert()
    {
        object result = "not a number";
        try
        {
            return (int)result; // Will throw
        }
        catch
        {
            return 0; // Default value
        }
    }

    // 4. Use appropriate generic type
    public void TypeMatching()
    {
        // Match return type to database column type
        // INT database column -> int return type
        // DECIMAL -> decimal
        // VARCHAR -> string
        // BIT -> bool
    }

    private T ExecuteScalar<T>(string query, object parameters = null)
    {
        // Implementation
        return default;
    }
}

```

Interview Questions

Dapper Questions

Q1: What are the main advantages of Dapper over Entity Framework?

A: Dapper is lightweight, extremely fast (close to raw [ADO.NET](#)), and gives full control over SQL queries. It has zero learning curve for SQL developers. EF adds abstraction which can hurt performance and generate inefficient queries. Use Dapper when speed is critical, EF when development speed matters.

Q2: How do you handle multiple result sets with Dapper?

A: Use QueryMultiple method that returns GridReader object with multiple results.

```
using (var reader = connection.QueryMultiple(query))
{
    var users = reader.Read<User>();
    var posts = reader.Read<Post>();
}
```

Q3: What is the difference between QueryFirstOrDefault and QuerySingleOrDefault in Dapper?

A: QueryFirstOrDefault returns first row or null. QuerySingleOrDefault throws if multiple rows exist, otherwise returns single row or null.

Q4: How does Dapper map database columns to object properties?

A: By convention, using exact property name matching (case-insensitive). You can customize with [Column] attribute or provide column-to-property mappings.

Entity Framework Questions

Q1: What is the difference between lazy loading, eager loading, and explicit loading?

A: Lazy loading loads related data when accessed. Eager loading uses Include() to load upfront. Explicit loading gives manual control with Load(). Choose based on query needs.

Q2: What is the DbContext and why is it important?

A: DbContext is the gateway to the database. It tracks changes, manages transactions, and translates LINQ to SQL. It implements Unit of Work pattern.

Q3: How does Entity Framework handle concurrent updates to the same record?

A: Use concurrency tokens. Add timestamp column or use RowVersion (byte[]). EF throws DbUpdateConcurrencyException if update fails.

Q4: What is the difference between SaveChanges() and SaveChangesAsync()?

A: SaveChanges() blocks thread. SaveChangesAsync() is non-blocking, better for high-concurrency scenarios (APIs, web servers).

JWT Questions

Q1: Why is JWT better than session-based authentication for APIs?

A: JWT is stateless (no server-side storage needed), scalable across servers, works well with microservices, and is ideal for mobile/SPA. Sessions require sticky sessions in distributed systems.

Q2: What happens if someone tampers with a JWT token?

A: The signature validation fails. If you change payload, signature doesn't match (you can't resign without secret key). Invalid signature is rejected immediately.

Q3: How do you handle token expiration?

A: Include exp claim with expiration time. Validate on each request. After expiration, user must log in again. Optionally implement refresh tokens for better UX.

Q4: Can JWT tokens be revoked?

A: Not directly (stateless). Solutions: token blacklist, short expiration time, or combination with refresh tokens. With short exp and refresh tokens, revocation becomes manageable.

IEnumerable vs IQueryable Questions

Q1: When would using IEnumerable instead of IQueryable cause performance issues?

A: If you have large database tables. IEnumerable loads ALL records into memory, then filters. IQueryable filters at database level. With 1 million records, wrong choice loads entire table.

Q2: Explain deferred execution with IEnumerable and IQueryable.

A: Both use deferred execution. Query isn't executed until you iterate (foreach) or call terminal operator (ToList, First, Count). This allows chaining and optimization.

Q3: Can you convert IQueryable to IEnumerable? What happens?

A: Yes, by calling ToList() or AsEnumerable(). After conversion, remaining operations execute in-memory. Use ToList() only when you want to materialize results.

Q4: Which methods return IQueryable vs IEnumerable?

A: DbSet returns IQueryable. List.AsQueryable() returns IQueryable. Arrays return IEnumerable. Collections return IEnumerable.

Ref vs Out Questions

Q1: When should you use ref vs out?

A: Use ref to modify existing variable. Use out to return multiple values or with TryXxx patterns (TryParse, TryGetValue). Out requires assignment, ref doesn't.

Q2: Can you pass null as ref parameter?

A: No, ref must be assigned before passing. You'd need to create a variable first.

Q3: What's the difference between passing by reference and passing by value?

A: By value creates copy. By reference passes the address. Changes to ref/out parameter affect original variable in calling code.

Q4: Show an example where out is more suitable than a return value.

A: TryParse pattern. Return bool for success/failure, use out for parsed value. Cleaner than returning tuple or exception-based handling.

Boxing vs Unboxing Questions

Q1: Why is boxing expensive?

A: It allocates memory on heap and copies the value. Multiple boxing/unboxing in tight loops kills performance.

Q2: How do generics help avoid boxing?

A: List stores actual int values, no boxing. ArrayList stores objects, boxing every int. Generics are faster and type-safe.

Q3: What happens if you try to unbox to wrong type?

A: InvalidCastException at runtime. If boxed as int, you must unbox to int (or nullable int). Can't unbox int to double.

Q4: Name scenarios where boxing happens implicitly.

A: ArrayList.Add(5), string concatenation with value types, IComparable interface, passing value type to object parameter, Dictionary with object values.

ExecuteScalar Questions

Q1: When should you use ExecuteScalar vs ExecuteReader?

A: ExecuteScalar for single value queries (COUNT, SUM, MAX, single ID). ExecuteReader for multiple columns or rows. ExecuteScalar is more efficient for scalar results.

Q2: How do you handle NULL values returned from ExecuteScalar?

A: Check for DBNull.Value or use nullable types with Dapper (ExecuteScalar<int?>). null coalescing operator ?? for default values.

Q3: What SQL commands are typical for ExecuteScalar?

A: Aggregate functions (COUNT, SUM, AVG, MAX, MIN), EXISTS check, SCOPE_IDENTITY for inserted ID, single value SELECT.

Q4: Compare ExecuteScalar in ADO.NET vs Dapper vs EF.

A: ADO.NET is verbose, returns object (type casting needed). Dapper is clean, generic type support. EF uses LINQ (Count(), Sum(), Max(), etc.). EF is most C#-idiomatic.

Mixed Scenario Questions

Q1: Build a user repository that inserts a user and returns the new ID using Dapper.

```
public int InsertUser(User user)
{
    using (var connection = new SqlConnection(_connectionString))
    {
        var query = @"INSERT INTO Users (UserName, Email)
                      VALUES (@UserName, @Email);
                      SELECT CAST(SCOPE_IDENTITY() as int)";

        return connection.ExecuteScalar<int>(query, user);
    }
}
```

Q2: Write a check to see if email exists using Entity Framework.

```
public bool EmailExists(string email)
{
    return _context.Users.Any(u => u.Email == email);
}
```

Q3: Implement JWT authentication in a controller method.

```
[HttpPost("login")]
public IActionResult Login(LoginRequest request)
```

```

{
    // Validate credentials
    if (ValidateCredentials(request.Username, request.Password))
    {
        var token = _jwtService.GenerateToken(userId, userName, email);
        return Ok(new { token });
    }
    return Unauthorized();
}

[HttpGet("protected")]
[Authorize]
public IActionResult Protected()
{
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    return Ok(new { message = $"User {userId} authenticated" });
}

```

Q4: Explain when you'd use ref vs out in a data validation method.

```

// Use out for validation with return value
public bool ValidateAndParseAge(string input, out int age)
{
    return int.TryParse(input, out age) && age > 0;
}

// Use ref to modify existing object
public void UpdateUserFromDatabase(ref User user, int userId)
{
    user = _context.Users.Find(userId);
}

```

Quick Reference Cheat Sheet

Dapper Methods

- Query() - Multiple rows
- QueryFirstOrDefault() - First or null

- QuerySingleOrDefault() - Single or null
- Execute() - No return
- ExecuteScalar() - Single value

Entity Framework Methods

- Add() / AddAsync() - Insert
- Update() - Modify
- Remove() - Delete
- SaveChanges() / SaveChangesAsync()
- Include() - Eager load
- Where() - Filter
- ToList() / ToListAsync() - Execute

JWT Claims

- iss (issuer)
- sub (subject/user ID)
- aud (audience)
- exp (expiration)
- iat (issued at)
- Custom claims

IEnumerable vs IQueryable

- IEnumerable: In-memory, LINQ to Objects
- IQueryable: Out-of-memory, provider-translated
- IQueryable inherits IEnumerable
- ToList() materializes IQueryable to IEnumerable

Ref vs Out

- ref: Initialize before, can read/write
- out: Need not initialize, must assign
- out: Better for TryXxx patterns
- ref: Better for modifying existing values

Boxing

- Happens: ArrayList, object parameter, string concat
- Avoid: Use generics, List instead of ArrayList

- Cost: Memory + performance
- Unboxing: Must match original type

ExecuteScalar Return Types

- int: COUNT, identity
 - decimal: SUM, AVG, amount
 - string: Single value
 - int?: Nullable for NULL handling
 - bool: EXISTS checks
-

Conclusion

Mastering these concepts is crucial for C# and .NET development:

- Dapper for high-performance data access
- Entity Framework for rapid development
- JWT for secure API authentication
- IEnumerable/IQueryable for correct filtering
- ref/out for advanced parameter passing
- Boxing/Unboxing understanding for performance
- ExecuteScalar for efficient scalar queries

Practice implementing each pattern. Real-world applications often combine multiple approaches. Choose the right tool for each specific scenario.